



Taller de Práctica: Base de Datos de un E-commerce (MySQL Avanzado)

Este taller de práctica está alineado estrictamente con el modelo relacional del proyecto de E-commerce (Productos, Categorías, Proveedores, Clientes, Ventas, Detalle_Ventas). Contiene 5 ejercicios espejo que permiten a los estudiantes afianzar la lógica requerida para los exámenes finales.

1. Evento Programado - Desactivación Automática de Productos sin Rotación

Descripción del caso de negocio

Para mantener el catálogo optimizado y evitar ofertar artículos que ya no tienen demanda, la tienda necesita descontinuar automáticamente aquellos productos que no hayan registrado ninguna venta en el último año (365 días), liberando espacio operativo en la plataforma.

Tarea

Crea un evento programado llamado `evt_desactivar_productos_obsoletos`.

- El evento debe ejecutarse de forma automática el primer día de cada mes a la medianoche.
- Debe añadir un campo `fecha_ultima_venta` (DATETIME, NULL por defecto) a la tabla `Productos`.
- Debe crear un trigger auxiliar `trg_actualizar_fecha_ultima_venta` que, tras insertarse un registro en `Detalle_Ventas`, actualice automáticamente el campo `fecha_ultima_venta` del producto correspondiente con la fecha de la venta o `NOW()`.



- El evento debe buscar aquellos productos activos (activo = VERDADERO) cuya fecha_ultima_venta sea anterior a hace 1 año (o productos creados hace más de 1 año que nunca hayan tenido ventas) y actualizar su campo activo a FALSO.
- Asegúrate de incluir la instrucción que habilita el planificador de eventos de MySQL (event_scheduler = ON).

Resultado esperado

Script .sql con:

- Sentencias ALTER TABLE para agregar las columnas necesarias a Productos.
- Sentencia CREATE TRIGGER para mantener sincronizada la fecha_ultima_venta.
- Sentencia CREATE EVENT con la periodicidad mensual y la lógica de desactivación.

Comentarios explicativos detallando el flujo.

2. Trigger - Auditoría de Cambios de Precios y Costos en Productos

Descripción del caso de negocio

Por motivos de control financiero y auditoría interna, la gerencia de la tienda exige un registro inmutable de cualquier alteración en el precio de venta o en el costo de adquisición de los productos del catálogo, registrando los valores previos, los nuevos y el margen resultante.

Tarea

Implementa un trigger llamado trg_audit_producto_after_update que se ejecute después de una actualización (AFTER UPDATE) en la tabla Productos.

- Crea una tabla de auditoría llamada Auditoria_Precios_Productos con los siguientes campos: id_auditoria (PK, AI), id_producto, campo_modificado



(VARCHAR), valor_antiguo (DECIMAL), valor_nuevo (DECIMAL), usuario (VARCHAR) y fecha_modificacion (DATETIME).

- El trigger debe activarse si cambia el precio o si cambia el costo.
- Si ambos valores cambian simultáneamente en una sola instrucción UPDATE, el trigger debe registrar una fila independiente en Auditoria_Precios_Productos por cada campo alterado.

Resultado esperado

- Sentencia CREATE TABLE para la tabla de auditoría Auditoria_Precios_Productos.
- Sentencia CREATE TRIGGER con validaciones condicionales independientes usando OLD y NEW.

Comentarios explicativos.

3. Procedimiento Almacenado - Cancelación Transaccional de Venta y Restauración de Stock

Descripción del caso de negocio

Cuando un pedido en estado 'Pendiente de Pago' o 'Procesando' es cancelado por el cliente o por el sistema de pagos, la tienda debe asegurar que la venta cambie formalmente de estado y que todas las unidades de los productos involucrados se reintegren inmediatamente al inventario, evitando discrepancias de stock.

Tarea

Crea un procedimiento almacenado llamado sp_CancelarVenta que reciba como parámetro de entrada id_venta_in y el motivo de cancelación (motivo_in).

- Validación: Comprobar que la venta exista y que su estado actual no sea ni 'Cancelado' ni 'Entregado'. Si la venta ya fue cancelada o entregada, debe generar un error explícito mediante SIGNAL SQLSTATE y no realizar modificaciones.



- Restauración de inventario: Iterar o actualizar en batch el stock en la tabla Productos, sumando la cantidad de cada producto registrado en Detalle_Ventas correspondiente a esa orden.
- Actualización de estado: Cambiar el campo estado de la venta a 'Cancelado'.
- Bitácora de cancelación: Insertar un registro en una tabla Cancelaciones_Ventas (id_cancelacion, id_venta, motivo, fecha_cancelacion).
- Atomicidad y control de errores: Todo el procedimiento debe ejecutarse dentro de una transacción explícita (START TRANSACTION, COMMIT) con un manejador de excepciones (DECLARE EXIT HANDLER FOR SQLEXCEPTION) que realice ROLLBACK ante cualquier falla.

Resultado esperado

- Sentencia CREATE TABLE para Cancelaciones_Ventas.
- Sentencia CREATE PROCEDURE con control transaccional estricto y manejo de errores.
- Comentarios explicando la atomicidad de la operación.

4. Función Definida por el Usuario - Cálculo del Margen de Beneficio por Cliente

Descripción del caso de negocio

El equipo de finanzas requiere conocer la contribución neta real de cada cliente a las ganancias del negocio, tomando en cuenta las compras realizadas, los costos de los productos y excluyendo las ventas canceladas.

Tarea

Crea una función SQL escalar llamada fn_CalcularMargenNetoCliente que reciba como parámetro id_cliente_in INT.

- Debe calcular el margen generado por el cliente sumando para todas sus ventas con estado 'Entregado' o 'Enviado': $\sum ((\text{precio_unitario_congelado} - \text{costo}) * \dots)$



cantidad). (Nota: vinculando Ventas, Detalle_Ventas y Productos para obtener el costo).

- La función debe retornar un valor DECIMAL(12,2) con la ganancia neta total.
- Si el cliente no registra ventas efectivas, debe manejar los valores nulos (COALESCE o IFNULL) y retornar 0.00.

Resultado esperado

- Sentencia CREATE FUNCTION optimizada y determinista con cálculo de agregación.
- Comentarios explicativos sobre el manejo de estados de venta y valores por defecto.

5. Consulta Avanzada - Clasificación ABC de Productos por Volumen de Ingresos (Ley de Pareto)

Descripción del caso de negocio

La administración de la tienda de E-commerce necesita clasificar todo el catálogo de productos según su contribución a las ventas globales históricas para priorizar las estrategias de abastecimiento y negociación con proveedores (Análisis ABC):

- Clase A: Productos que representan hasta el 80% del valor total acumulado de ventas.
- Clase B: Productos que representan entre el 80% y el 95% del acumulado.
- Clase C: Productos que representan el 5% final (entre el 95% y el 100%).

Tarea

Escribe una única consulta SQL analítica que realice lo siguiente:

- Agrupar por id_producto, nombre y categoria para calcular el total monetario facturado histórico: SUM(cantidad * precio_unitario_congelado).
- Calcular el porcentaje de participación de cada producto sobre la facturación global de la tienda utilizando funciones de ventana (SUM(...) OVER()).



- Calcular el porcentaje acumulado ordenado de forma descendente por ventas (SUM(...) OVER (ORDER BY total_ventas DESC)).
- Asignar la categoría resultante ('Clase A', 'Clase B', 'Clase C') mediante una estructura condicional CASE.

Resultado esperado

- Consulta SQL única estructurada con Common Table Expressions (CTE WITH) y funciones de ventana analíticas.
- Comentarios detallando cómo se calcula la ventana acumulada y la segmentación.